

Lab6_Report

Cliff P.

Edge Impulse Link : [here](#)

Part 1

raw inputs :

time series data (windowed acceleration in X, Y, Z coordinate)

NN classifier features :

type : spectral

number of input features : 33

feature names (5): accX RMS, accX Peak 1 Height, accZ Spectral Power 2.0-5.0, accY Peak 3 Height, accY RMS

NN classifier outputs : nominal, off, anomaly score

Anomaly detector features :

type : spectral

number features: 33

feature names: accX RMS, accX Peak 1 Height, accY Spectral Power 2-5 Hz, accZ Spectral Power 2-5 Hz

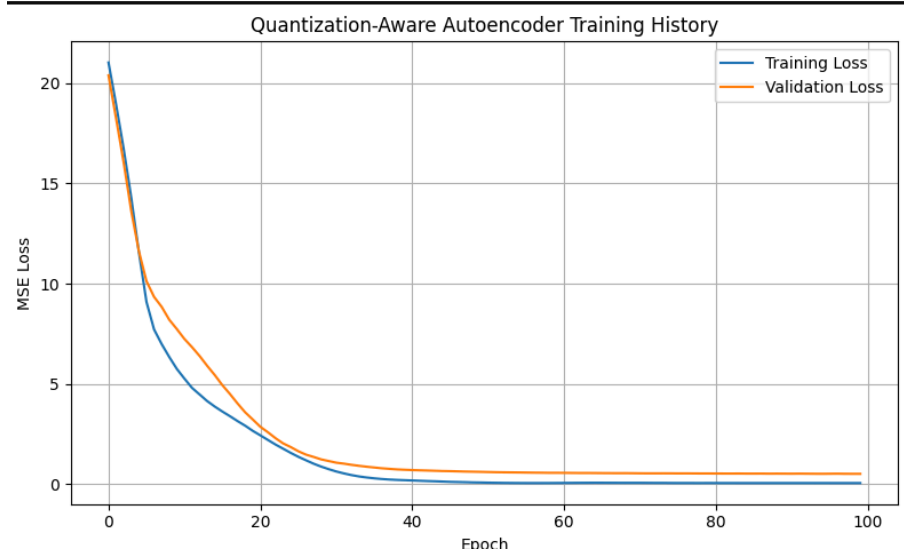
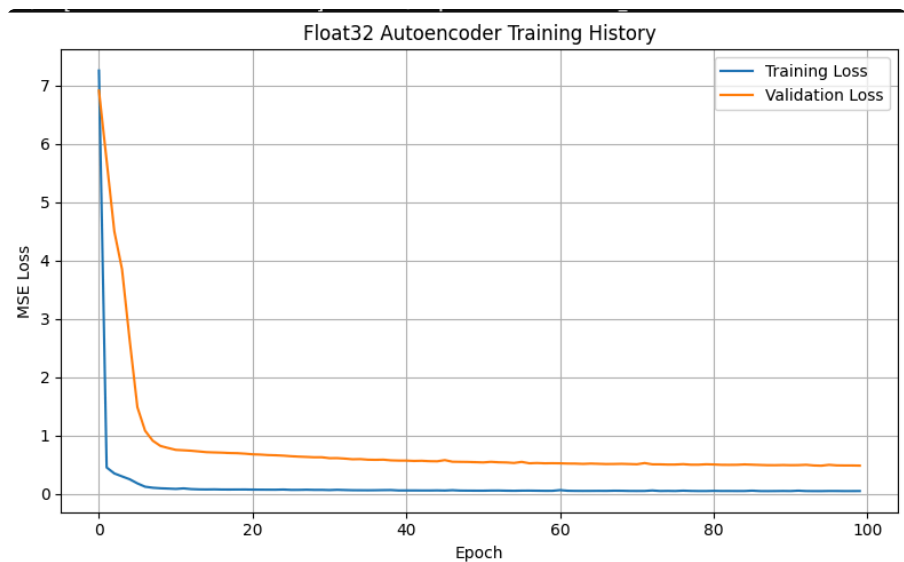
Deployment Evidence :

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 609 us, anomaly 506 us, postprocessing 5 ms
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 16.022089
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 610 us, anomaly 6 ms, postprocessing 50 us
#Classification predictions:
  nominal: 0.996094
  off: 0.000000
Anomaly prediction: 400.567047
```

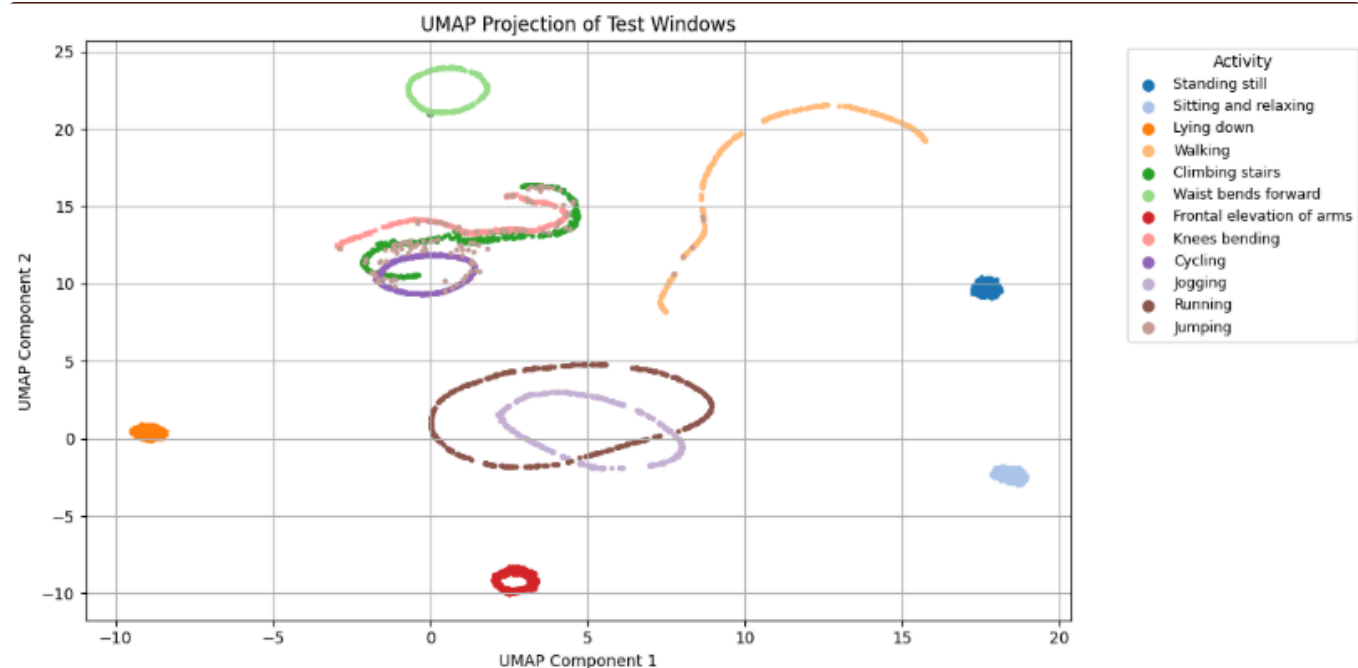
when board is sat still, anomaly prediction is low, however when board is shaken, anomaly prediction increases. Should not always believe prediction values as board shaken could be from outside factors that aren't malicious.

Part 2

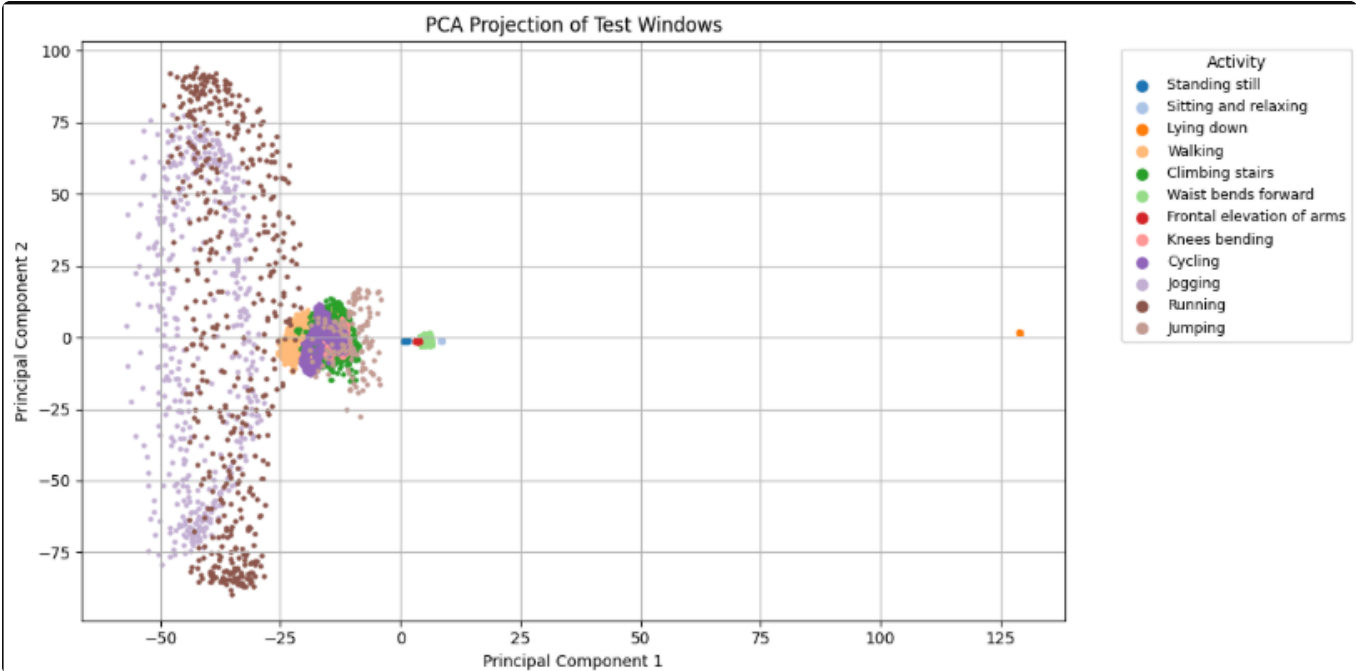
Training and Validation loss curve



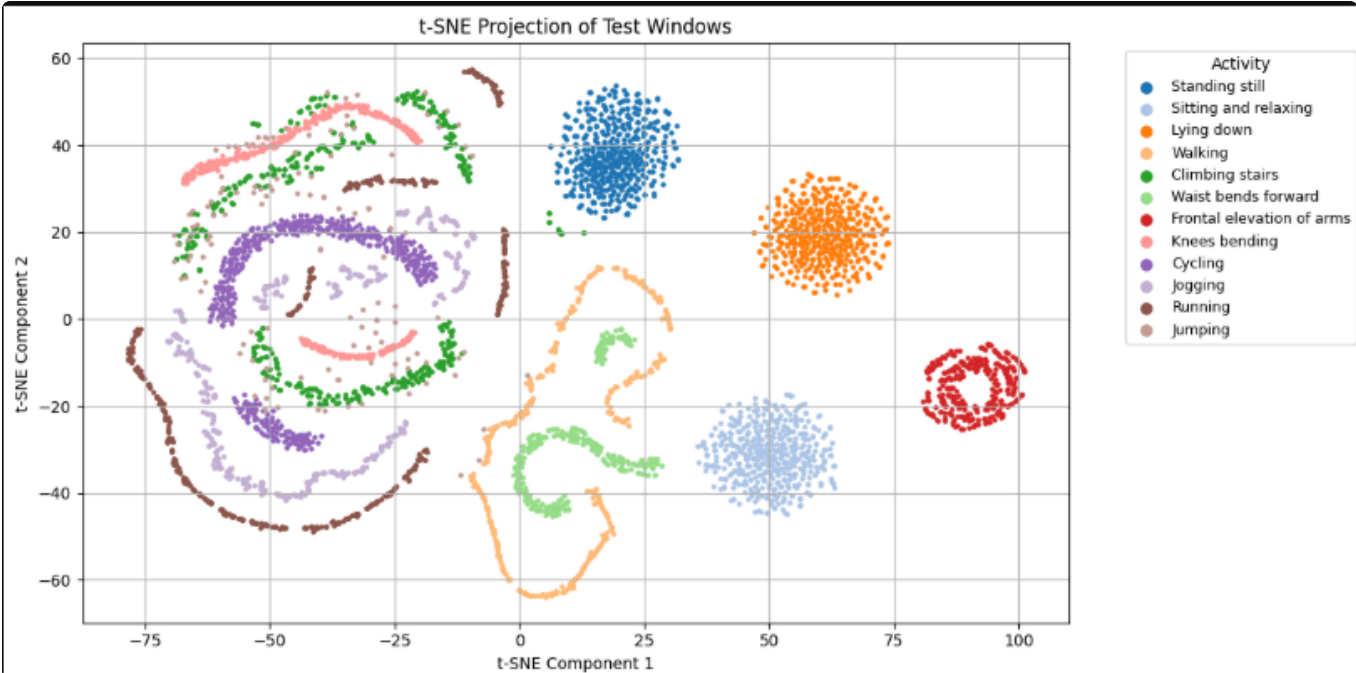
UMAP Projection



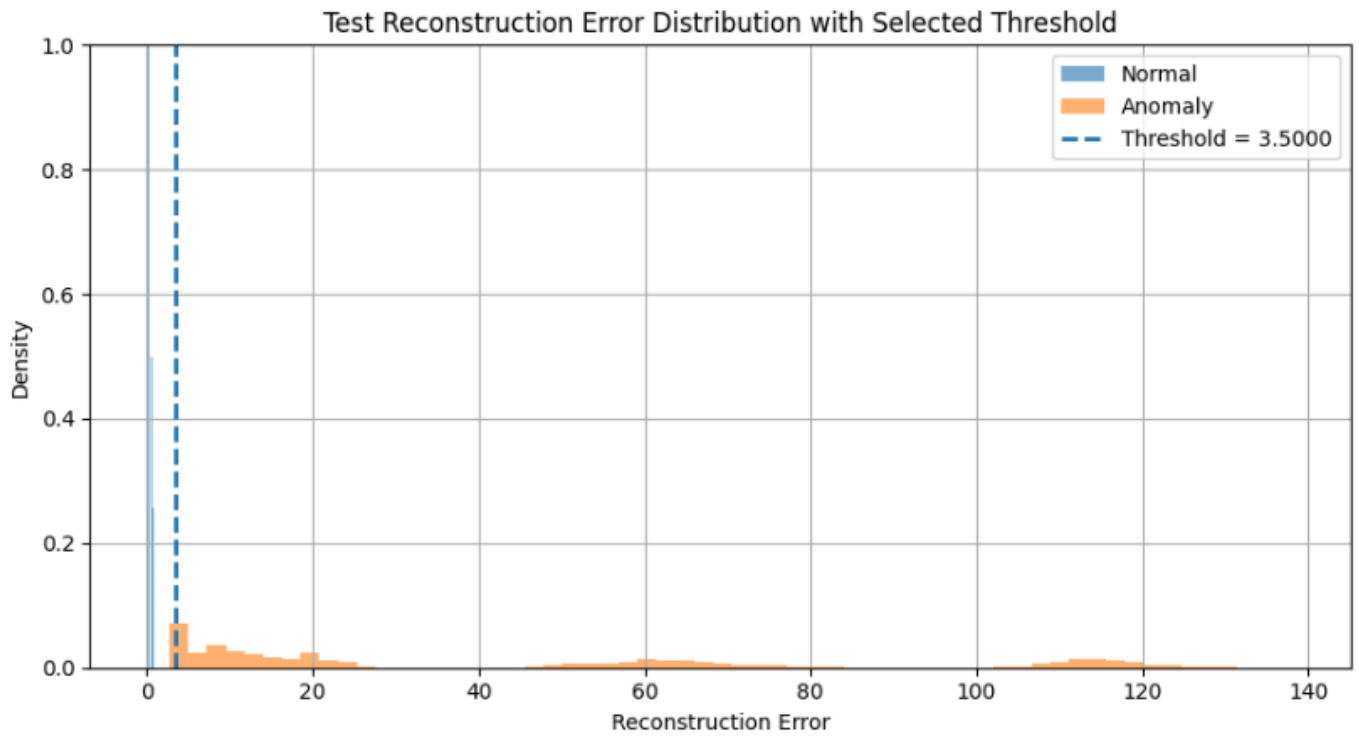
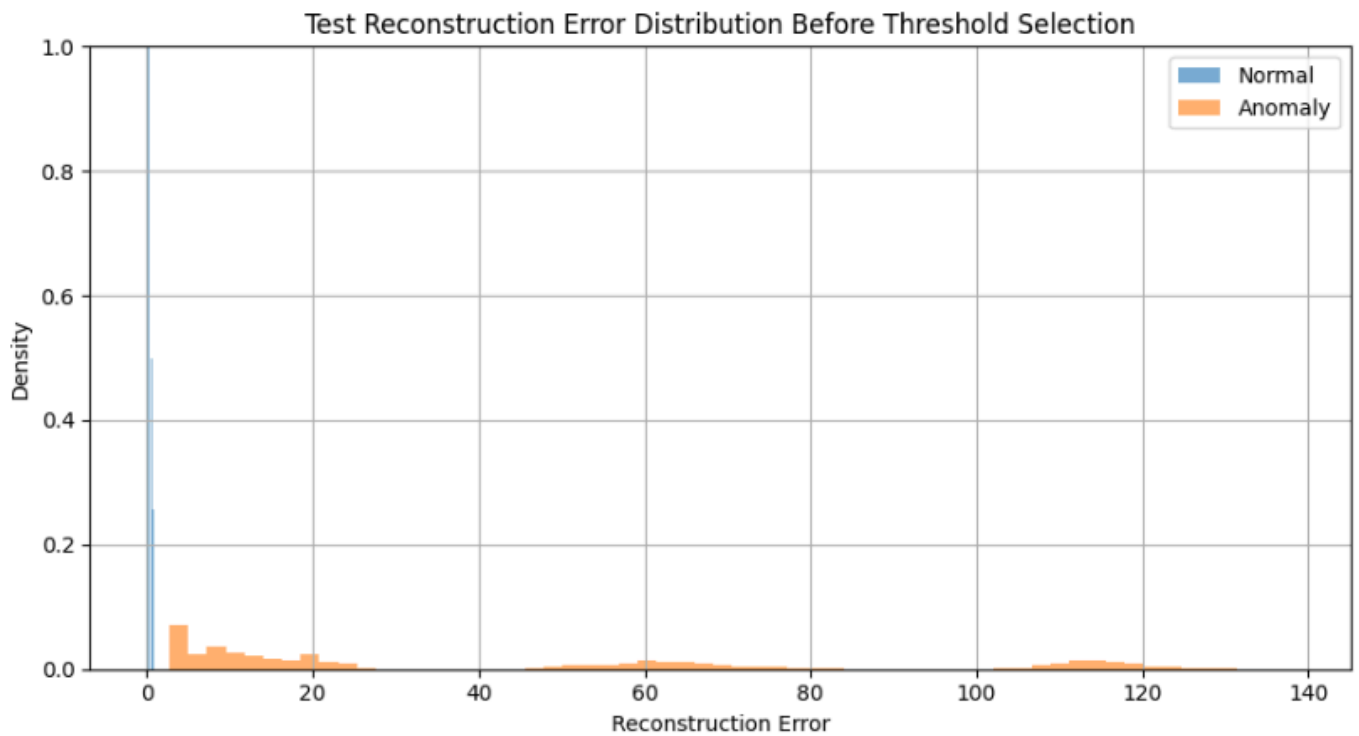
PCA Projection



t-SNE Projection

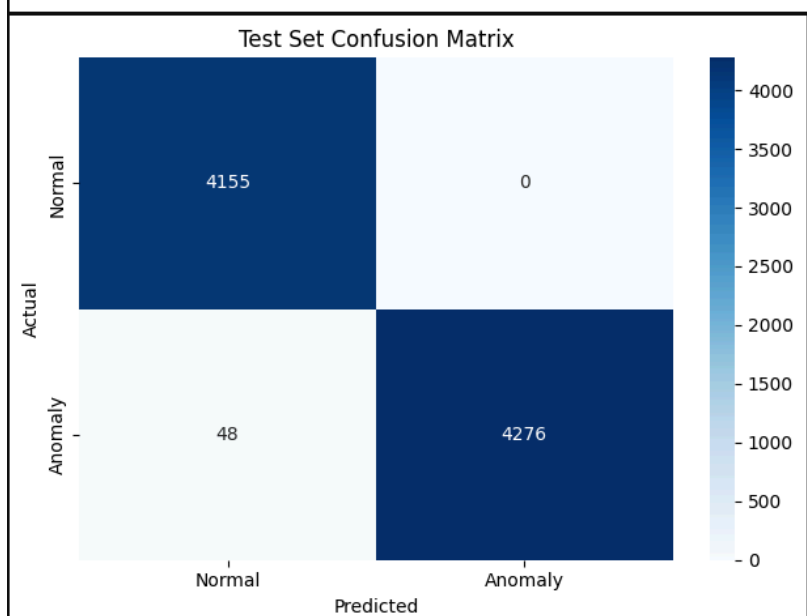
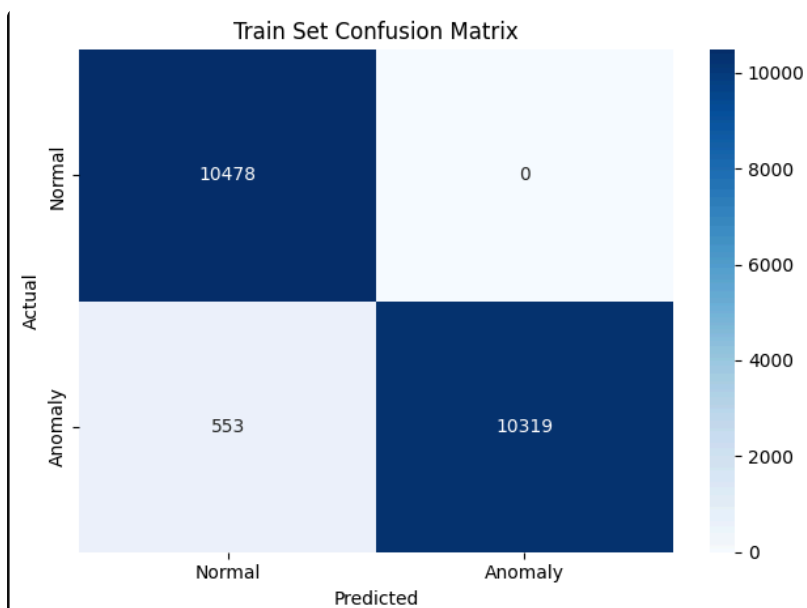


Error Distribution (Threshold = 3.5)



Evaluation Metric & Confusion Matrix

Train set evaluation				
	precision	recall	f1-score	support
Normal (0)	0.95	1.00	0.97	10478
Anomaly (1)	1.00	0.95	0.97	10872
accuracy			0.97	21350
macro avg	0.97	0.97	0.97	21350
weighted avg	0.98	0.97	0.97	21350
Test set evaluation				
	precision	recall	f1-score	support
Normal (0)	0.99	1.00	0.99	4155
Anomaly (1)	1.00	0.99	0.99	4324
accuracy			0.99	8479
macro avg	0.99	0.99	0.99	8479
weighted avg	0.99	0.99	0.99	8479



Serial Monitor of reconstruction error and anomaly and normal detection

```
Reconstruction error: 1.324718
Result: Normal activity
Reconstruction error: 1.364547
Result: Normal activity
Reconstruction error: 1.337564
Result: Normal activity
Reconstruction error: 1.471637
Result: Normal activity
Reconstruction error: 1.503215
Result: Anomaly detected
Reconstruction error: 1.510451
Result: Anomaly detected
Reconstruction error: 1.561115
Result: Anomaly detected
```

Discussion Questions

1. I chose a threshold of 3.5 by observing the reconstruction error distribution, I see the normal activity peaks around 1.5 and ends near 3.0. The anomaly starts being more stronger at around 3.5, hence I chose 3.5 as it was a value that best fit where the normal and anomaly start separating clearly.
2. Reconstruction error help distinguish normal vs anomaly activities because the autoencoder is trained initially with the normal activities, meaning it can reconstruct the normal activities very closely to the original normal activity input. Now if we feed in an anomaly activity that is very different from an expected normal activity input, the autoencoder will result in a high reconstruction error as it was not trained to reconstruct anomaly activities, which can be a good indicator that our input is an anomaly.
3. The information from the notebook that must be perserved while deploying to the Arduino is
 1. Window Size
 2. Feature Count and Order
 3. Quantization Parameter because this is an int8 model, so the input scale and the zero point must be used to convert the raw floats into integers back.
4. Important for the sketch to use the same preprocessing assumptions as the notebook because we want the same variables and metrics across both platforms to ensure no indeterministic behaviors which can stem from if we chose different metrics or parameters / other factors that are different between the notebook and the sketch. We would then get unexpected results from the Arduino deployment than what we were expecting after training the model on the notebook.

5. Main limitations of this anomaly detection using autoencoders on microcontrollers:

1. Deep autoencoders can consume a lot of memory / storage
2. If permanent factors changes such as wearing different shoes or walking on different surface, then the model cannot adapt instantly to those factors and must be retrained and redeployed.